恒生电子股份有限公司

前端编码规范

说　明

本文档中所包含的信息属于商业机密信息，如无恒生电子股份有限公司的书面许可，任何人都无权复制或利用。

编辑部门：EPG 研发组

批准日期：2022/10/20

密级：内部

目录

1 引言 3

2 约束等级定义 3

3 环境要求 3

4 编码规范 4

4.1 HTML/TEMPLATE 开发规范 4

4.1.1 通用语法 4
4.1.2 HTML5 Doctype 6
4.1.3 语言属性 7
4.1.4 字符编码 7
4.1.5 引入 CSS 和 JavaScript 文件 7
4.1.6 减少标签的数量 7
4.1.7 属性顺序 8
4.1.8 语义化 8
4.1.9 注释 9
4.1.10 多媒体回溯 9

4.2 CSS/LESS/SASS 开发规范 9

4.2.1 命名 9
4.2.2 语法 10
4.2.3 代码风格 15
4.2.4 样式兼容性 17
4.2.5 选择器权重（样式覆盖） 17
4.2.6 声明简写 18
4.2.7 css 动画 19
4.2.8 声明顺序 19
4.2.9 字体规则 20
4.2.10 hack 规范 20

4.3 JS/ES6 开发规范 21

4.3.1 命名 21
4.3.2 语法 22
4.3.3 代码风格 28
4.3.4 数组、对象 30
4.3.5 使用 null 31
4.3.6 文档注释 31

4.4 VUE 开发规范 33

4.4.1 vue 命名规范 33
4.4.2 文件夹命名规范 35
4.4.3 语法 35
4.4.4 组件/实例的选项的顺序 41
4.4.5 元素特性的顺序 42
4.4.6 单文件组件的顶级元素的顺序 42

4.5 前端请求调用规范 43

5 文档修改记录 44

1 引言

本文旨在为前端开发人员提供统一的编码规范，可以更快、更高效、更规范的完成繁重、复杂、多样化的任务。制作规范的主要目的在于：

- 降低每个组员介入项目的门槛成本；
- 提高工作效率及协同开发的便捷性；
- 高度统一的代码风格；

2 约束等级定义

约束等级	约束效力	强制性
【强制】	违反该项将被认为代码存在严重缺陷	前端程序必须遵守
【推荐】	违反该项将被认为代码存在轻微缺陷	根据具体产品特性的不同，选择性地遵守
【参考】	违反该项可被认为代码存在优化空间	从产品持续优化及人员技能提升的角度，参考使用

3 环境要求

1) 【推荐】使用 Chrome 浏览器，若是使用 Vue.js 技术栈开发，推荐安装 Vue.js devtools 插件进行调试
2) 【推荐】使用 **Node.js LTS** 版本，可以使用 nvm 或 nvm-windows 在一台电脑中管理多个 Node 版本
3) 【推荐】使用 **Visual Studio Code (VS Code)** 进行代码编写
4) 【推荐】安装插件 Prettier - Code formatter（ CSS / Less / JS 等其他文件代码格式化；Vetur 的格式化基于此插件实现，故可以在所有文件实现统一的格式化）
5) 【推荐】规定 VS Code 中设置 Tab 大小为 2 个空格，保证在所有环境下获得一致展现

```
// vscode->settings.json
{
    "editor.tabSize": 2,
    // ...
}
```

4 编码规范

4.1 HTML/TEMPLATE 开发规范

4.1.1 通用语法

1) 【强制】缩进使用两个空格代替 Tab

说明：前端代码层级较深，使用短缩进有利于利用屏幕空间，提升效率

使用两个空格代替 Tab 可以保证在所有环境下获得一致展现

```
<!--  not good-->
<div>
    <div>bar</div>
</div>

<!--  good -->
<div>
  <div>bar</div>
</div>
```

2) 【强制】嵌套元素应当缩进一次（即两个空格），同层级缩进应保持一致

```
<!--  not good-->
<div>
<div>bar</div>
    <div>bar</div>
</div>

<!--  good -->
<div>
  <div>bar</div>
  <div>bar</div>
</div>
```

3) 【强制】将每个块元素、列表元素或表格元素都放在新行；

- inline 元素视情况换行，以长度不超过编辑器一屏为宜；
- 每个子元素都需要相对其父级缩进；

4) 【强制】对于属性的定义，使用双引号，不要使用单引号

```
<!--  not good-->
<input class='a' type=text>

<!--  good -->
<input class="a" type="text">
```

5) 【强制】不要省略可选的结束标签（closing tag）（如`</li>` 或 `</body>`）

说明：省略可选的结束标签，虽不会违反 H5 规范；但可能会造成层级上的困扰，导致代码出现无法预料的问题

```
<!--  not good-->
<h1>h1 text
  <h2>h2 text

<!--  good -->
<h1>h1 text</h1>
<h2>h2 text</h2>
```

6) 【强制】特殊符号使用 HTML 字符实体（实体名称对大小写敏感），常用如下：

| 符号 | 实体编码 | | :--: | :------: | | 空格 | \ | | © | \© | | ¥ | \¥ | | ® | \® | | > | \> | | < | \< | | & | \& |

7) 【强制】td / th 要在 tr 里面，li 要在 ul / ol 里面

```
<!--  not good-->
<table>
  <td>table</td>
</table>

<!--  good -->
<table>
  <tr>
    <td>table</td>
  </tr>
</table>
```

8) 【强制】ul / ol 的直接子元素只能是 li，不能包含其他元素

```
<!--  not good-->
<ul>
  <span>span</span>
  <li>a</li>
  <li>b</li>
</ul>
```

9) 【强制】行内元素里面不可使用块级元素

说明：a 标签是一个行内元素，行内元素里面套了一个 div 的标签，这样可能会导致 a 标签无法正常点击

```
<!--  not good-->
<!--  not good-->
<a href="../test">
  <div></div>
</a>
```

可以使用如下代码进行修复：

```
<!--  good -->
```

```
<a href="../test" style="display: block">
  <div></div>
</a>
```

10) 【强制】不使用重复属性，重复的属性只会取第一个

```
<!--  error-->
<input type="text" class="a" class="b">

<!-- good -->
<input type="text" class="a b">
```

11) 【强制】不要在 https 的链接里写 http 的图片

说明：只要 https 的网页请求了一张 http 的图片，就会导致浏览器地址栏左边的小锁没有了，一般不要写死，写成根据当前域名的协议去加载，用//开头：

```
<img src="//static.chimeroi.com/hello-world.jpg">
```

12) 【强制】所有元素必须正确嵌套,不允许交叉嵌套

13) 【推荐】链接与锚点

说明：在 a 标签中的 href，没有具体指向链接时，应该使用“javascript:void(0)”，而不是“#”，避免导致页面不必要的跳转；需要在页内跳转时，请使用 #id 的方式

14) 【推荐】不要在自闭合（self-closing）元素的尾部添加斜线

说明：HTML 中一些元素是没有封闭标签的，应保持其用法， HTML5 规范中说明这是可选的，元素如：area， base， br， col， command， embed， hr， img， input， keygen , link， meta， param， source， track， wbr

15) 【推荐】不使用属性设置样式（`img`, `table` 等元素）

```
<!-- not good -->
<img src="test.jpg" alt="test" width="400" height="200">

<!-- good -->
<img src="test.jpg" alt="test" style="width: 400px; height: 200px;">
```

16) 【推荐】文件或类中的私有属性，变量和方法名应该以下划线 "_" 开头；保护属性，变量和方法名不需要下划线开头，和公共变量名一样。建议结合 ES6 的私有属性规范，用 # 作为类的私有属性修饰符。

17) 【参考】自定义属性要以 data-开头 自己添加的非标准的属性要以 data-开头，否则 w3c validator 会认为是不规范的

```
<!—  not good—>
<div count="5"></div>

<!—  good —>
<div data-count="5"></div>
```

4.1.2 HTML5 Doctype

1) 【强制】为每个 HTML 页面添加标准模式（standard mode）的声明，确保在每个浏览

器中拥有一致的展现

```
<!DOCTYPE html>
<html lang="zh-CN">
...
```

4.1.3 语言属性

1) 【强制】为每个 HTML 页面根元素添加 lang 属性

说明：根据 HTML5 规范： 强烈建议为 html 根元素指定 lang 属性，从而为文档设置正确的语言。这将有助于语音合成工具确定其所应该采用的发音，有助于翻译工具确定其翻译时所应遵守的规则等等。

4.1.4 字符编码

1) 【强制】通过声明一个明确的字符编码，让浏览器轻松、快速的确定网页内容渲染方式，通常指定为'UTF-8'

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="utf-8">
。。。
```

2) 【强制】标签书写

说明：所有的 HTML 标签必须小写；所有的 HTML 属性必须小写；

4.1.5 引入 CSS 和 JavaScript 文件

1) 【强制】根据 HTML5 规范，在引入 CSS 和 JavaScript 文件时不需要指定 `type` 属性，因为 `text/css` 和 `text/javascript` 分别是它们的默认值

2) 【推荐】脚本加载放在 body 之后

说明：浏览器在解析 HTML 时，是按照标签进行顺序加载解析的，脚本引入如果放在 head 中，会阻塞 DOM 的加载解析，在设计上脚本主要用于处理交互逻辑的，不应该影响首次呈现过程，同时脚本文件的引入也应该放在 HTML 内容（body）的之后。

建议将脚本的引入放在 body 尾部。

3) 【推荐】样式加载放在 head 中

说明：样式对 DOM 的渲染影响机制是先声明的优先生效，建议放在内容（body）之前进行加载（head）。样式资源本身建议规划为基础样式、全局样式、UI 样式。

4.1.6 减少标签的数量

1) 【推荐】编写 HTML 代码时，尽量避免多余的层级

```
<!--  not good-->
<span class="avatar">
  <img src="..." alt="...">
</span>

<!--  good -->
<img class="avatar" src="..." alt="...">
```

4.1.7 属性顺序

1) 【参考】属性应该按照特定的顺序出现以保证易读性

```
● class
● id
● name
● data-*
● src,for,type,href,value,max-length,max,min,pattern
● placeholder,title,alt
● aria-*,role
● required,readonly,disabled
```

4.1.8 语义化

1) 【参考】尽量遵循 HTML 标准和语义

说明：HTML 的语法中包含的标签（元素），很多是具备语义化的，从规范上建议优先使用具有语义的标签，这样可以提高代码的可读性、代码复用和执行效率，但是不要以牺牲实用性为代价；任何时候都要尽量使用最少的标签并保持最小的复杂度。

```
<!-- The page header should go into a header element -->
<header>
<!-- As this title belongs to the page structure it's a heading and h1 should be
used-->
  <h1>My page title</h1>
</header>
<!-- All navigation should go into a nav element -->
<nav class="top-navigation">
 <!-- A listing of elements should always go to UL (OL for ordered listings) -->
  <ul>
     <li class="nav-item"><a href="#home">Home</a></li>
     <li class="nav-item"><a href="#news">News</a></li>
     <li class="nav-item"><a href="#about">About</a></li>
   </ul>
</nav>
```

4.1.9 注释

1) 【推荐】尽可能的为代码写上注释，解释为什么要这样写，解决了什么问题等，在每个模块开始和结束的地方添加注释

```
<!-- 列表模块 -->
<div class="list">
    …
</div>
<!-- /列表模块 -->
```

2) 【推荐】中英文之间、中文和数字之间用空格隔开。

4.1.10 多媒体回溯

1) 【推荐】对于页面上的媒体而言，如图片、视频、canvas 动画，建议增加可替代的识别 UI，如图片可以使用 alt 增加文字说明，alt 属性的内容为对该图片的简要描述，这对于盲人用户和图像损毁都非常有意义，即无障碍。对于纯粹的装饰性图片，alt 属性值可以留空，如 alt=""；视频和音频可以增加字幕和文字说明，能够真实描述内容而不是用途。

4.2 CSS/LESS/SASS 开发规范

4.2.1 命名

1) 【强制】类名及 id、关键帧命名可以使用大写字母、小写字母，允许以中划线(“-”) 或者下划线(“_”)进行分隔

```
// foo-bar
// FooBar
// foo-bar--foo-bar
// foo-bar__foo-bar
// FooBar--FooBar
// FooBar__FooBar
// FooBar--foo-bar
// foo-bar--FooBar
// Foo-bar
```

2) 【强制】不允许使用拼音，尤其是缩写的拼音、拼音与英文的混合
3) 【强制】命名-词汇规范
   - 不依据表现形式来命名；
   - 可根据内容来命名；
   - 可根据功能来命名。

   不推荐：left, right, center, red, black

推荐：nav, aside, news, type, search

4) 【强制】命名-缩写规范

- 保证缩写后还能较为清晰保持原单词所能表述的意思；
- 使用业界熟知的或者约定俗成的 ；

不推荐：

navigation => navi

header => head

description => des

推荐：

navigation => nav

header => hd

description => desc

5) 【强制】定义某个样式名，都有同名的样式已存在的可能性，因此尽量写上前缀，减少全局污染。

6) 【强制】避免 class 与 id 重名，id 用于标识模块或页面的某一个父容器区域，名称必须唯一，不要随意新建 id

7) 【推荐】less/sass 中的变量、函数、混合等采用驼峰式命名

```
// less
@mianFontColor: #444;

#companyName,
.company-name {
  Color: @mainFontColor;
}

// sass
$mianFontColor: #444;

#{$companyName},
.company-name {
  Color: @mainFontColor;
}
```

4.2.2 语法

1) 【强制】所有声明语句都应当以分号结尾

说明：最后一条声明语句后面的分号是可选的

```
// error
.selector {
  font-size: 15px
  Color: #000
}
```

```
// not good
.selector {
  font-size: 15px;
  color: #000
}

// good
.selector {
  font-size: 15px;
  color: #000;
}
```

2) 【强制】避免为 0 值指定单位（长度单位是：em、ex、ch、vw、vh、cm、mm、in、pt、pc、px、rem、vmin 和 vmax），例如，用 margin: 0; 代替 margin: 0px;
3) 【强制】不出现空的规则（声明块中没有声明语句）
4) 【强制】禁止未知的伪类选择器
5) 【强制】禁止未知的伪元素选择器
6) 【强制】为选择器中的属性添加双引号，例如，input[type="text"]； 某些情况下是可选的，但是，为了代码的一致性，都加上双引号

```
// not good
.selector[type=text] {
  font-size: 15px
  Color: #000
}

// good
.selector[type="text"] {
  font-size: 15px;
  color: #000
}
```

7) 【强制】十六进制值必须全部小写，例如，#f3f6fa
8) 【强制】禁止使用无效的十六进制颜色
9) 【强制】伪类选择器必须为小写格式，例如，a:hover {} :root {}
10) 【强制】伪元素选择器必须为小写格式，例如，a:before a::before
11) 【强制】函数名称必须为小写，例如，a:hover { width: calc(5% - 10em); }
12) 【强制】指定 media 媒体功能名称必须为小写格式
13) 【强制】类型选择器必须为小写格式
14) 【强制】属性必须为小写格式
15) 【强制】指定字符串必须使用双引号
16) 【强制】禁止未知的单位
17) 【强制】禁止在同一规则中由于另一个属性值而忽略的属性值
18) 【强制】指定的 @ 规则名必须小写格式
19) 【强制】指定关键字的值必须为小写格式

```
// not good
a {
  display: BLOCK;
}

// good
a {
  display: block;
}
```

20) 【强制】不要设置太大的 z-index，最小值为 1，最大值为 100（但一个正常的系统的层级关系在 10 以内就能完成，除了在复写 UI 组件库某个组件的时候）
21) 【强制】禁止使用未知或者错误的属性，例如，a { my-property: 1; }
22) 【强制】禁止数字尾随零 1.0/0.50，例如，a { top: 1.010000px; }
23) 【强制】禁止未知的@规则名称,但为了兼容 scss、less 等预处理器，排除一些特殊字段，

```
["error", {
"ignoreAtRules": [
    "content",
    "each",
    "else",
    "extend",
    "for",
    "function",
    "if",
    "include",
    "import",
    "mixin",
    "rule"
]
}]
```

24) 【强制】禁止在 linear-gradient() 中调用不符合标准语法的无效方向值
25) 【强制】URL 必须有引号
26) 【强制】换行符必须始终为 LF（\n）

27) 【强制】禁止关键帧声明的 !important
28) 【强制】在声明的块中禁止出现重复的属性
29) 【强制】禁止在样式表中使用重复的 @import 规则
30) 【强制】禁止样式表中的重复选择器
31) 【强制】禁止出现未知的 media 特性名称
32) 【强制】禁止 media 特性名称带有浏览器引擎前缀
33) 【强制】禁止简写属性覆盖相关的扩写属性
34) 【强制】相邻空行数量为 2
35) 【强制】注释规范
    1) 多写注释，且多使用句子进行描述而不是词语
    2) 禁止出现空的注释
    3) 禁止出现在注释中的单词黑名单

```
"/^TODO:/",
"/^FIXME:/",
"fuck",
"shit",
"damn",
"twerk",
"egg yolk"
```

    4) 保持注释内容与星号之间有一个空格的距离

普通注释（单行）：

```
/* 普通注释 */
```

区块注释：

```
/*
* 模块: m-box
* 描述
* 应用：page detail, info and etc...
*/
```

36) 【强制】选择器应该在满足功能的基础上尽量简短，减少选择器嵌套，一般选择器不要超过 5 层（在 Less 中避免嵌套超过 5 层）

37) 【强制】使用简写形式的十六进制值，例如，用 #fff 代替 #ffffff
38) 【强制】禁止使用命名颜色，例如，用 #000 代替 black
39) 【强制】禁止在 `calc` 函数内使用无效的表达式
40) 【强制】禁止在 `calc` 函数内使用不加空格的操作符
41) 【强制】禁止使用 CSS 不支持并可能导致意外结果的双斜杠注释 (//...)

```
// not good
a {
  //color: #000;
}
//a { color: #000; }

// good
a {
  /* color: #000; */
}
```

42) 【强制】对于属性值或颜色参数，省略小于 1 的小数前面的 0 （例如，.5 代替 0.5；-.5px 代替 -0.5px）
43) 【推荐】用 border: 0; 代替 border: none;
44) 【推荐】合并 margin、padding、border 的-left/-top/-right/-bottom 的设置，尽量使用短名称
45) 【推荐】当有效的关键字（normal、bold、bolder 和 lighter）可用时，font-weight 的值推荐使用关键字，而不是数字
46) 【推荐】不要使用*选择器，一个选择器中通用选择器的最大数量为 1
47) 【推荐】一个选择器中属性选择器的最大数量为 8
48) 【推荐】一个选择器中的最大数量为 8
49) 【推荐】一个选择器中复合选择器的最大数量为 8
50) 【推荐】适当使用:before 和:after 来画页面的一些视觉上的辅助性元素，如三角形、短的分隔线、短竖线等，可以减少页面上没有用的标签
51) 【推荐】推荐使用相对协议的链接

```
// not good
a {
  background: url("//www.google.com/file.jpg");
}

// good
a {
  background: url("//www.google.com/file.jpg");
}
a {
  background: url("../file.jpg");
}
a {
  background: url("http://www.google.com/file.jpg");
}
```

```
a {
  background: url("/path/to/file.jpg");
}
```

4.2.3 代码风格

注意：此处大部分工作将由代码格式化工具或者代码检查工具完成（参见环境要求），一般无需考虑

1) 【强制】缩进使用两个空格代替 Tab
2) 【强制】为选择器分组时，将单独的选择器单独放在一行

```
// not good
.selector, .selector-secondary, .selector[type=text] {
  /* ... */
}

// good
.selector,
.selector-secondary,
.selector[type="text"] {
  /* ... */
}
```

3) 【强制】声明块的左花括号前添加一个空格；关闭前不能有空格
4) 【强制】声明块的右花括号应当单独成行，必须有一个换行符；右花括号关闭前必须有一个换行符
5) 【强制】每条声明语句的 : 后应该插入一个空格, : 前不加空格
6) 【强制】声明前禁止空行

```
// not good
    a {
      --foo: #000;

      bottom: 15px;
    }
    a {

      bottom: 15px;

      top: 5px;
    }
// good
a {
  --foo: #000;
  bottom: 15px;
```

```
}
a {
  bottom: 15px;
  top: 5px;
}
```

7) 【强制】声明的属性值包含行时，则冒号后必须换行符

```
// not good
a {
 box-shadow: 0 0 0 1px #5b9dd9,
    0 0 2px 1px rgba(30, 140, 190, 0.8);
}

// good
a {
  box-shadow:
    0 0 0 1px #5b9dd9,
    0 0 2px 1px rgba(30, 140, 190, 0.8);
}
```

8) 【强制】每条样式声明应该独占一行

```
// not good
.selector {
  font-size: 15px; color: red;
}

// good
.selector {
  font-size: 15px;
  color: red;
}
```

9) 【强制】对于以逗号分隔的属性值，每个逗号后面都应该插入一个空格（例如，box-shadow，transition）

```
// not good
.selector {
  transition: border .2s,color .3s,padding: .4s;
}

// good
.selector{
  transition: border .2s, color .3s, padding: .4s;
}
```

10) 【强制】!important 前插入一个空格，后不能有空白符

11) 【强制】注释：//后插入一个空格，/*后插入一个空格，*/前插入一个空格

12) 【强制】less/sass 的操作符，在圆括号中的数学计算表达式的数值、变量和操作符之间均添加一个空格
13) 【强制】具有多行参数时，@ 规则名必须有一个换行符
14) 【强制】在 @ 规则名之前必须有空行；之后必须有一个空格
15) 【强制】在 @ 规则的分号之后必须有一个换行符；之前禁止有空白符
16) 【强制】在多行函数的逗号之后禁止有一个空白符；之前禁止有一个空白符
17) 【强制】在单行函数的逗号之后必须有一个空格
18) 【强制】限制函数中相邻空行的数量为 0
19) 【强制】禁止行尾空格
20) 【强制】禁止出现额外的分号
21) 【强制】在组合选择器之前后都必须有一个空格
22) 【强制】禁止后代选择器使用非空格字符
23) 【强制】注释内必须保留一个空格
24) 【推荐】注释统一用/* */

4.2.4 样式兼容性

1) 【强制】当使用一些较新的 CSS3 语法时，不需要添加浏览器前缀（在打包时由 postcss 处理）
2) 【推荐】不要使用 input 的 line-height 来做垂直居中 设置 line-height 为一个很高的值会导致 Safari 浏览器的输入光标变得巨大 (与 line-height 等高)

```
// not good
input {
  height: 40px;
  line-height: 40px;
}

// good
input {
  height: 20px;
  line-height: 20px;
  padding: 10px 0;
}
```

4.2.5 选择器权重（样式覆盖）

权重的基本规则：

1. 相同的权重：以后面出现的选择器为最后规则
2. 不同的权重，权重值高则生效

详细了解权重计算方法

1) 【强制】非通用样式使用嵌套方式进行编写，避免影响其他自己不了解样式，造成

样式覆盖

2) 【强制】禁止在具有较高优先级的选择器后出现被其覆盖的较低优先级的选择器

3) 【强制】无法修改原样式声明时，应通过权重关系，编写权重更高的样式进行覆盖

4) 【强制】不使用 !important，除非原样式使用内联样式或 !important 且无法直接修改

5) 【推荐】Vue 中样式谨慎使用 scoped，会影响样式选择器性能，请使用第一点进行特有样式编写

6) 【推荐】样式需要修改时，尽量找到原样式声明进行修改

4.2.6 声明简写

1) 【推荐】当你不确定自己写的属性会否影响到其他属性时，应避免使用简写

```
// not good
.selector{
  margin: 0 0 10px;
  background: #000;
  background-image: url("image.jpg");
  border-radius: 3px 3px 0 0;
}

// good
.selector{
  margin-bottom: 10px;
  background-color: #000;
  background-image: url("image.jpg");
  border-top-left-radius: 3px;
  border-top-right-radius: 3px;
}
```

2) 【推荐】当你确定自己的声明不会影响到其他属性时，请使用简写提升代码简洁性

```
// not good
.selector {
  padding-top: 10px;
  padding-right: 20px;
  padding-bottom: 15px;
  padding-left: 20px;
}

// good
.selector {
 padding: 10px 20px 15px 20px;
}
```

4.2.7 css 动画

1) 【强制】不要使用 all 属性做动画

说明：使用 transition 做动画的时候不要使用 all 所有属性，在有一些浏览器上面可能会有一些问题，如下：

```
transition: all 2s linear;
```

在 Safari 上面可能会有一些奇怪的抖动，正确的做法是要用哪个属性做动画就写哪个，如果有多个就用隔开，如下代码所示：

```
transition: transform 2s linear, opacity 2s linear;
```

2) 【推荐】位移动画使用 transform 替代 position （提升动画性能）
3) 【推荐】使用 CSS 动画替代 JS 动画

4.2.8 声明顺序

1) 【参考】相关的属性声明按以下顺序做分组处理，组之间需要有一个空行，可以使用 stylelint-order
   A. **Positioning**（影响其他元素和自身位置相关声明）：display/list-style/position/float/ clear/ z-index/zoom/overflow …
   B. **Box model**（自身盒模型相关声明）：width/height/margin/padding/border/ line-height…
   C. **Typographic**（文本相关声明）：color/font/text-decoration/text-align/text-indent/ vertical-align/white-space/content…
   D. **Visual**（自身样式）background/border-radius/cursor…
   E. **Misc**（其他声明）:CSS3 属性 transform/transition/animation/box-shadow

   注意：

   使用 CSS3 的属性，如果有必要加入浏览器前缀，则按照 -webkit- / -moz- / -ms- / -o- / std 的顺序进行添加，标准属性写在最后。

   链接的样式请严格按照如下顺序添加： a:link -> a:visited ->a:hover -> a:active

```
.declaration-order {
  /* Position */
  display: block;
  position: absolute;
  top: 0;
  right: 0;
  bottom: 0;
  left: 0;
  z-index: 10;
  float: right;
```

```
  /* Box-model */
  width: 100px;
  height: 100px;
  border: 1px solid #000;

  /* Typography */
  font: normal 13px "Helvetica Neue", sans-serif;
  line-height: 1.5;
  color: #333;
  text-align: center;

  /* Visual */
  background-color: #000;
  border-radius: 3px;

  /* Misc */
  opacity: 1;
};
```

4.2.9 字体规则

为了防止文件合并及编码转换时造成问题，建议将样式中文字体名字改成对应的英文名字，如：黑体(SimHei)、 宋体(SimSun) 、微软雅黑(Microsoft Yahei)，几个单词中间有空格组成的必须加引号 。字体粗细采用具体数值，粗体 bold 写为 700，正常 normal 写为 400。font-size 必须以 px 或 pt 为单位，推荐用 px（注：pt 为打印版字体大小设），不允许使用 xx-small/x-small/small/medium/large/x-large/xx-large 等值 。为了对 font-family 取值进行统一，更好的支持各个操作系统上各个浏览器的兼容性，font-family 不允许在业务代码中随便添加。

1) 【强制】指定字体名称当推荐使用引号时再使用引号
2) 【强制】禁止使用重复的字体名称
3) 【推荐】推荐在字体系列名称列表中使用通用系列
4) 【推荐】当你不确定自己写的属性会否影响到其他属性时，应避免使用简写

4.2.10 hack 规范

1) 【推荐】尽可能的减少对 Hack 的使用和依赖，如果在项目中对 Hack 的使用太多太复杂，对项目的维护将是一个巨大的挑战；
2) 【推荐】使用其它的解决方案代替 Hack 思路；
3) 【推荐】如果非 Hack 不可，选择稳定且常用并易于理解的。

```
.test {
  color: #000; /* For all */
  color: #111\9; /* For all IE */
```

```
  color: #111\0; /* For IE8 and later, Opera without Webkit */
  color: #333\9\0; /* For IE8 and later */
  color: #444\0; /* For IE8 and later */
  *color: #666; /* For IE7 and earlier */
  _color: #777; /* For IE6 and earlier */
}
```

4.3 JS/ES6 开发规范

4.3.1 命名

1) 【强制】支持 ES6 语法的文件中变量全部采用 let 定义，常量使用 const 定义，（let/const 属于 ES6 语法，当浏览器不支持 ES6 语法时，需 babel 转义，不支持 ES6 语法时变量采用 var 定义
2) 【强制】标准变量采用驼峰式命名（考虑与后台交换数据的情况，对象属性可灵活命名）
3) 【强制】常量全大写，用下划线连接
4) 【强制】变量名不应过短，要能准确完整地描述该变量所表述的事物

不好的变量名	好的变量名
inp	input, priceInput
day1, day2, param1	today, tomorrow
id	userId, orderId
obj	orderData, houseInfos
tId	removeMsgTimerId
handler	submitHandler,searchHandler

5) 【强制】变量名不要使用计算机术语，如 texareaData，应该取和业务相关的名字，如 leaveMsg
6) 【强制】变量名的对仗要明确，如 up/down、begin/end、opened/closed、visible/invisible、scource/target
7) 【强制】变量名使用正确的语法

说明：不要使用中文拼音，如 shijianchuo 应改成 timestamp ； 如果是复数的话加 s，或者加上 List，如 orderList、menuItems； 而过去式的加上 ed，如 updated/found 等； 如果正在进行的加上 ing，如 calling；

8) 【推荐】文件或类中的私有属性，变量和方法名应该以下划线 "_" 开头；保护属性，变

量和方法名不需要下划线开头，和公共变量名一样。

9) 【推荐】使用临时变量时请结合实际需要进行变量命名

说明：有些喜欢取 temp 和 obj 之类的变量，如果这种临时变量在两行代码内就用完了，接下来的代码就不会再用了，还是可以接受的，如交换数组的两个元素。但是有些人取了个 temp，接下来十几行代码都用到了这个 temp，这个就让人很困惑了。所以应该尽量少用 temp 类的变量

```
// not good
let temp = 10;
let leftPosition = currentPosition + temp,
    topPosition = currentPosition - temp;

// good
let adjustSpace = 10;
let leftPosition = currentPosition + adjustSpace,
    topPosition = currentPosition - adjustSpace;
```

10) 【推荐】波尔变量可以结合实际语境使用 done/found/success/ok/available/complete/is 等修饰词

```
// good
let ajaxDone = true;
let fileFound = false;
let resourceUpdated = false;
```

11) 【推荐】波尔变量名应使用肯定的布尔变量名，不要使用否定的名词，如 notOk、notReady，因为否定的词取反的时候就会比较奇怪，如 if (!notOk)

4.3.2 语法

1) 【强制】变量不要先使用后声明
2) 【强制】不要声明了变量却不使用
3) 【强制】不要在同一个或者父作用域下声明同名变量
4) 【强制】一个函数作用域中所有的变量声明尽量提到函数首部

```
// not good
let registerForm = null,
  question = '',
  calculateResult = 0;

// good
let registerForm = null;
let question = '';
let calculateResult = 0;
```

5) 【强制】为了快速知晓变量类型，声明变量时要赋值

```
// not good
let registerForm;
```

```
let question;
let calculateResult;

// good
let registerForm = null;
let question = '';
let calculateResult = 0;
```

6) 【强制】单一函数的返回值类型要确定（如下无法确定该函数的最终返回类型）
7) 【强制】setter 必须有对应的 getter，getter 可以没有对应的 setter
8) 【强制】getter 必须有返回值，并且禁止返回空；禁止 setter 有返回值
9) 【强制】setter 和 getter 必须写在一起
10) 【强制】constructor 中必须有 super；禁止在 super 被调用之前使用 this 或 super
11) 【强制】禁止方向错误的 for 循环
12) 【强制】禁止 throw 字面量，必须 throw 一个 Error 对象
13) 【强制】禁止在 catch 中仅仅只是把错误 throw 出去，这样的 catch 是没有意义的，等价于直接执行 try 里的代码
14) 【强制】禁止使用保留字作为变量名
15) 【强制】禁止使用未定义的变量
16) 【强制】禁止将 undefined 赋值给变量
17) 【强制】循环内必须对循环条件中的变量有修改
18) 【强制】禁止在 in 或 instanceof 操作符的左侧变量前使用感叹号
19) 【强制】禁止在普通字符串中出现模版字符串里的变量形式
20) 【强制】变量须先定义后使用，方法除外（考虑到递归的情况）;已定义的变量必须使用
21) 【强制】函数赋值给变量的时候，函数名必须与变量名一致
22) 【强制】禁止函数的循环复杂度超过 20，参考 Cyclomatic_complexity
23) 【强制】debugger 不要出现在提交的代码里
24) 【强制】禁止在 return, throw, break 或 continue 之后还有代码
25) 【强制】禁止在 finally 中出现 return, throw, break 或 continue，
    说明：finally 中的语句会在 try 之前执行
26) 【强制】代码块嵌套的深度禁止超过 5 层
27) 【强制】回调函数嵌套禁止超过 3 层，多了请用 async await 替代
28) 【强制】函数的参数禁止超过 3 个
29) 【强制】new 后面的类名必须首字母大写
30) 【强制】禁止将 async 函数做为 new Promise 的回调函数
    说明：出现这种情况时，一般不需要使用 new Promise 实现异步了
31) 【强制】当使用 new 关键字调用没有参数的构造函数时，该规则禁止使用括号
32) 【强制】禁止对一个变量使用 delete，编译阶段就会报错了
33) 【强制】禁止使用 caller 或 callee，它们是已废弃的语法
34) 【强制】for in 内部必须有 hasOwnProperty

```
// not good
for (key in foo) {
    doSomething(key);
}
```

```
// good
for (key in foo) {
    if (Object.prototype.hasOwnProperty.call(foo, key)) {
        doSomething(key);
    }
}
```

35) 【强制】switch 的 case 内有变量定义的时候，必须使用大括号将 case 内变成一个代码块
36) 【强制】switch 语句中的 default 必须在最后
37) 【强制】禁止在函数参数中出现重复名称的参数，严格模式下，编译阶段会报错
38) 【强制】禁止对已定义的 class 重新赋值
39) 【强制】禁止重复定义类的成员
40) 【强制】申明后不再被修改的变量必须使用 const 来申明
41) 【强制】Promise 的 reject 中必须传入 Error 对象，而不是字面量
42) 【强制】优先使用正则表达式字面量，而不是 RegExp 构造函数
43) 【强制】禁止 if else 的条件判断中出现重复的条件
44) 【强制】禁止在对象字面量中出现重复的键名
45) 【强制】禁止在 switch 语句中出现重复测试表达式的 case
46) 【强制】禁止重复导入模块
47) 【强制】禁止出现空代码块，允许 catch 为空代码块
48) 【强制】禁止在正则表达式中使用空的字符集 []
49) 【强制】禁止解构赋值中出现空 {} 或 []
50) 【强制】parseInt 必须传入第二个参数
51) 【强制】generator 函数内必须有 yield
52) 【强制】禁止将 catch 的第一个参数 error 重新赋值
53) 【强制】禁止修改原生对象
    说明：修改原生对象可能会与将来版本的 js 冲突
54) 【强制】禁止与负零进行比较
55) 【强制】禁止在 setTimeout 或 setInterval 中传入字符串
56) 【强制】禁止使用 ~+ 等难以理解的类型转换，仅允许使用 !!
57) 【强制】禁止出现没必要的 bind、call 或 apply
58) 【强制】禁止出现没必要的计算键名，没必要的字符串连接，没必要的 constructor，没必要的转义

```
// not good
const foo = {
  ['1']: 1,
  ['bar']: 'bar'
};

// good
const foo = {
  1: 1,
  bar: 'bar'
};
```

```
// not good
const foo = 'f' + 'oo';
const bar = `b` + `ar`;

// good
const foo = 'fo';
const bar = 1 + `ar`;

// not good
class Foo {
  constructor() {}
}
class Bar extends Foo {
  constructor(...args) {
    super(...args);
  }
}

// good
class Foo {
  constructor() {
    doSomething();
  }
}
class Bar extends Foo {
  constructor(...args) {
    super(...args);
    doSomething();
  }
}
```

59) 【强制】禁止对导入的模块进行赋值
60) 【强制】禁止在 if 代码块内出现函数声明
61) 【强制】禁止不必要的布尔类型转换
62) 【强制】禁止在 RegExp 构造函数中出现非法的正则表达式
63) 【强制】禁止在类之外的地方使用 this
说明：只允许在 class 中使用 this
64) 【强制】小数点前后必须有数字
65) 【强制】禁止使用特殊空白符（比如全角空格），除非是出现在字符串、正则表达式或模版字符串中
66) 【强制】禁止使用 __iterator__
说明：__iterator__ 是一个已废弃的属性，使用 [Symbol.iterator] 替代它
67) 【强制】switch 的 case 内必须有 break, return 或 throw，空的 case 除外
68) 【强制】禁止使用超出 js 精度范围的数字

69) 【强制】禁止将自己赋值给自己，将自己与自己比较
70) 【强制】禁止使用逗号操作符
71) 【强制】换行时操作符必须在行尾，其中，？ ：可以放到行首
72) 【强制】禁止在 return 语句里赋值，除非用小括号包裹
73) 【强制】禁止正则表达式中使用肉眼无法区分的特殊字符
说明：某些特殊字符很难看出差异，最好不要在正则中使用
74) 【强制】不允许用混合空格和制表符表示缩进
75) 【强制】禁止使用 \ 来换行字符串
76) 【强制】禁止将 Math, JSON 或 Reflect 直接作为函数调用
77) 【强制】禁止使用 new Function、new Object、new require，使用 new 来生成 String,Number 或者 Boolean
78) 【强制】禁止直接 new 一个类而不赋值
说明：new 应该作为创建一个类的实例的方法，所以不能不赋值
79) 【强制】禁止在 if， else if， while 等表达式中使用赋值语句，除非这个赋值语句被括号包起来了
80) 【强制】禁止对使用 const 定义的常量重新赋值
81) 【强制】禁止对函数的参数重新赋值
82) 【强制】禁止重复定义变量
83) 【强制】禁止使用 __proto__
说明：__proto__ 是已废弃的语法
84) 【强制】禁止使用八进制的转义符，编译阶段就会报错了
85) 【强制】禁止使用 0 开头的数字表示八进制数，编译阶段就会报错了
86) 【强制】禁止将常量作为分支条件判断中的测试表达式，但允许作为循环条件判断中的测试表达式
87) 【强制】禁止在构造函数中返回值
88) 【强制】禁止使用 'strict'
89) 【强制】创建 Symbol 时必须传入参数
90) 【强制】必须使用 isNaN(foo) 而不是 foo === NaN
91) 【强制】必须使用 if (foo === 5) 而不是 if (5 === foo)
92) 【强制】typeof 表达式比较的对象必须是 'undefined', 'object', 'boolean', 'number', 'string', 'function', 'symbol', 或 'bigint'
93) 【强制】禁止解构赋值时出现同样名字的的重命名，比如 let { foo: foo } = bar;
94) 【强制】禁止对一个变量使用 delete，编译阶段就会报错了
95) 【强制】使用 let 定义变量，const 定义常量
96) 【强制】使用===代替==，!==代替!=（==会自动进行类型转换，可能会出现奇怪的结果）

```
null == undefined; // true
'' == '0'; // false
0 == ''; // true
0 == '0'; // true
'  \t\r\n' == 0; // true
new String('abc') == 'abc'; // true
new Boolean(true) == true; // true
true == 1; // true
```

97) 【推荐】使用三目运算代替简单的 if-else

```
// not good
let seatDiscount = 100;
if (seat < 5) {
  seatDiscount = 90;
} else if(seat < 10) {
  seatDiscount = 80;
} else {
  seatDiscount = 70;
}

// good
let seatDiscount = seat < 5 ? 90 : seat < 10 ? 80 : 70;
```

98) 【推荐】使用 ... 而不是 Object.assign，除非 Object.assign 的第一个参数是一个变量

99) 【推荐】使用箭头函数取代简单的函数（ES6 支持）

```
// not good
let that = this;
setTimeout(function () {
  that.foo = 'bar';
});

// good
setTimeout(() => {
  this.foo = 'bar';
});
```

100) 【推荐】在必要的地方添加非空判断以提高代码的稳健性

101) 【推荐】将复杂的函数分解成多个子函数，方便维护和复用

102) 【参考】用闭包有可能导致循环引用引起的 GC 无法释放内存，如：

```
function foo(element, a, b) {

    element.onclick = function() { /* uses a and b */ };

}
```

这里，即使没有使用 element，闭包也保留了 element、a 和 b 的引用，由于 element 也保留了对闭包的引用，这就产生了循环引用，这就不能被 GC 回收。这种情况下，可将代码重构为：

```
function foo(element, a, b) {

    element.onclick = bar(a, b);

}
```

function bar(a, b) {

return function() { /* uses a and b */ }

}

4.3.3 代码风格

注意：此处大部分工作将由代码格式化工具或者代码检查工具完成（参见环境要求），一般无需考虑

1) 【强制】缩进使用两个空格代替 Tab
2) 【强制】统一使用单引号或者反引号，除非有转义需要
3) 【强制】语句结尾需要加分号
4) 【强制】不允许结尾存在多余的逗号
5) 【强制】箭头函数的箭头前后需有括号
6) 【强制】逗号在行尾
7) 【强制】文件结尾处有一个空行
8) 【强制】最大连续空行不准超过 1
9) 【强制】在同一行中的关闭大括号和开始大括号内强制实施一致的间距
10) 【强制】扩展运算符和表达式之间不允许有空格
11) 【强制】generator 函数星号和 function 无空格，和函数名有空格
12) 【强制】类的成员之间需要空行，如果写在同一行跳过空行检查
13) 【强制】块的左大括号放在同一行及其相应的说明或声明后，要在同一行

```
// not good
function foo()
{
    return true;
}

if (foo)
{
    bar();
}

try
{
    somethingRisky();
} catch(e)
{
    handleError();
}

if (foo) {
```

```
    bar();
}
else {
    baz();
}

// good
function foo() {
    return true;
}

if (foo) {
    bar();
}

if (foo) {
    bar();
} else {
    baz();
}

try {
    somethingRisky();
} catch(e) {
    handleError();
}

function nop() { return; }

if (foo) { bar(); }

if (foo) { bar(); } else { baz(); }

try { somethingRisky(); } catch(e) { handleError(); }
```

14) 【强制】以下几种情况不需要空格：

- 对象的属性名后
- 前缀一元运算符后
- 后缀一元运算符前
- 函数调用括号前
- 无论是函数声明还是函数表达式，'('前不要空格
- 数组的'['后和']'前
- 对象的'{'后和'}'前
- 运算符'('后和')'前
- 数组括号和其他标记之间

- 逗号前无

15) 【强制】以下几种情况需要空格：
    - 二元运算符前后
    - 三元运算符'?:'前后
    - 代码块'{'前
    - 下列关键字前：else, while, catch, finally
    - 下列关键字后：if, else, for, while, do, switch, case, try,catch, finally, with, return,typeof
    - 单行注释'//'后（若单行注释和代码同行，则'//'前也需要），多行注释'*'后
    - 对象的属性值前
    - for 循环，分号后留有一个空格，前置条件如果有多个，逗号后留一个空格
    - 无论是函数声明还是函数表达式，'{'前一定要有空格
    - 函数的参数之间
    - 逗号后
    - 操作符左右两边
    - 注释的斜线或 * 后

4.3.4 数组、对象

1) 【强制】对象属性名不需要加引号
2) 【强制】对象以缩进的形式书写，不要写在一行
3) 【强制】数组中不要存在空元素
4) 【强制】禁止在数组中出现连续的逗号
5) 【强制】禁止使用 Array 构造函数时传入的参数超过一个
    - 参数为一个时表示创建一个指定长度的数组，比较常用
    - 参数为多个时表示创建一个指定内容的数组，此时可以用数组字面量实现，不必使用构造函数
6) 【强制】不要用 for in 循环数组
7) 【强制】数组的方法除了 forEach 之外，回调函数必须有返回值
8) 【强制】for-in 只用于 object/map/hash 的遍历。对 Array 用 for-in 循环有时会出错，因为它并不是从 0 到 length - 1 进行遍历，而是所有出现在对象及其原型链的键值。
9) 【推荐】数组、对象最后不要有逗号

```
// not good
let a = {
  'b': 1
};

let a = { b: 1 };

let a = {
  b: 1,
  c: 2,
}

// good
```

```
let a = {
  b: 1,
  c: 2
}
```

4.3.5 使用 null

1) 【强制】正确使用 null

适用场景：

- 初始化一个将来可能被赋值为对象的变量
- 与已经初始化的变量做比较
- 作为一个参数为对象的函数的调用传参
- 作为一个返回对象的函数的返回值
  1. 不要用 null 来判断函数调用时有无传参
  2. 不要与未初始化的变量做比较

```
// not good
function test(a, b) {
  if (b === null) {
    // not mean b is not supply
    // ...
  }
}

let a;

if (a === null) {
  // ...
}

// good
let a = null;

if (a === null) {
  // ...
}
```

4.3.6 文档注释

1) 【推荐】版权注释

```
/**
* 版权信息：Copyright ® 恒生电子股份有限公司
*/
```

2) 【推荐】属性注释

/**
* Maximum number of things per pane.
* @type {number}
*/
project.MyClass.prototype.someProperty = 4;

3) 【推荐】各类标签 @param, @method 等请参考 usejsdoc 和 JSDoc Guide；

建议在以下情况下使用：

- 所有常量
- 所有函数
- 所有类

```
/**
 * func
 * @desc 一个带参数的函数
 * @param {string} a
 * @param {number} b
 * @param {string} c
 * @param {object} d
 */
function fun(a, b, c, d) {
  // ...
}
```

4) 【推荐】插件注释

插件注释通常放在插件定义的最前面。通常详细介绍该插件的功能、作者、版本、以及产生该插件的时间和引用其他组件或聚合等信息。如下：

/**
* 表格组件,用于渲染普通表格
*
* 引用 jQuery.js,share.js
* 作者 AnonaWang
* 版本 0.1
* 时间 2019-06-01
*/

4.4 VUE 开发规范

4.4.1 vue 命名规范

1) 【强制】组件名应该始终是多个单词的，根组件 App 除外

```
// not good
Vue.component('todo', {
  // ...
})

export default {
  name: 'todoitem',
  // ...
}

// good
Vue.component('TodoItem', {
  // ...
})

export default {
  name: 'TodoItem',
  // ...
}
```

2) 【强制】组件名应该以高级别的 (通常是一般化描述的) 单词开头，以描述性的修饰词结尾

```
// not good
components/
  |- ClearSearchButton.vue
  |- ExcludeFormSearchInput.vue
  |- LaunchOnStartupCheckbox.vue
  |- RunSearchButton.vue
  |- SearchInput.vue
  |- TermsCheckbox.vue

// good
components/
  |- SearchButtonClear.vue
  |- SearchButtonRun.vue
  |- SearchInputQuery.vue
  |- SearchInputExcludeGlob.vue
  |- SettingsCheckboxTerms.vue
```

```
  |- SettingsCheckboxLaunchOnStartup.vue
```

3) 【强制】和父组件紧密耦合的子组件应该以父组件名作为前缀命名

说明：如果一个组件只在某个父组件的场景下有意义，这层关系应该体现在其名字上。因为编辑器通常会按字母顺序组织文件，所以这样做可以把相关联的文件排在一起

```
// not good
components/
  |- TodoList.vue
  |- TodoItem.vue
  |- TodoButton.vue
components/
  |- SearchSidebar.vue
  |- NavigationForSearchSidebar.vue

// good
components/
  |- TodoList.vue
  |- TodoListItem.vue
  |- TodoListItemButton.vue
components/
  |- SearchSidebar.vue
  |- SearchSidebarNavigation.vue
```

4) 【推荐】业务页面名称（非模板组件）统一使用小写字母开头的(camelCase)规范,例如：index.js/index.vue/bondItem.vue

5) 【推荐】应用特定样式和约定的基础组件 (也就是展示类的、无逻辑的或无状态的组件) 应该全部以一个特定的前缀开头，比如 Base、App 或 V

```
// not good
components/
  |- MyButton.vue
  |- VueTable.vue
  |- Icon.vue

// good
components/
  |- BaseButton.vue
  |- BaseTable.vue
  |- BaseIcon.vue

components/
  |- AppButton.vue
  |- AppTable.vue
  |- AppIcon.vue

components/
```

```
  |- VButton.vue
  |- VTable.vue
  |- VIcon.vue
```

6) 【推荐】组件名应该倾向于完整单词而不是缩写

```
// not good
components/
  |- SdSettings.vue
  |- UProfOpts.vue

// good
components/
  |- StudentDashboardSettings.vue
  |- UserProfileOptions.vue
```

4.4.2 文件夹命名规范

1) 【推荐】./src 下除 components 外的子文件夹命名均使用(camelCase)规范，例如 bizViews
2) 【推荐】./src/components 下最多只有一层子文件夹，文件夹名称为组件的名称,命名使用以大写字母开头的 PascalBase 风格，文件夹下含有 index.vue 或 index.js 例如 FaBox

4.4.3 语法

1) 【强制】组件的 data 必须是一个函数（除了 new Vue 外的任何地方）
2) 【强制】prop 的定义应该尽量详细，至少需要指定其类型

```
// not good
props: ['status']

// good
props: {
  status: String
}

// better
props: {
  status: {
    type: String,
    required: true,
    validator(value) {
      return [
        'syncing',
        'synced',
        'version-conflict',
```

```
        'error'
      ].indexOf(value) !== -1
    }
  }
}
```

3) 【强制】为 v-for 设置键值：在组件上总是必须用 key 配合 v-for，以便维护内部组件及其子树的状态在组件上总是必须用 key 配合 v-for，以便维护内部组件及其子树的状态
4) 【强制】计算属性禁止包含异步方法

```
// not good
export default {
    computed: {
        foo: async function () {
            return await someFunc();
        },
        bar() {
            return new Promise();
        },
        baz() {
            setTimeout(() => {}, 0);
        }
    }
};
```

5) 【推荐】禁用已废弃的 scope 、slot-scope 属性
6) 【推荐】使用 v-slot 替代已废弃的 slot
7) 【强制】禁止重复的键名
8) 【强制】禁止出现重复的属性
9) 【强制】组件必须包含 name 属性，且禁止使用保留字
10) 【强制】禁止覆盖保留字
11) 【强制】禁止在 <textarea> 中出现模版语法 {{message}}
12) 【强制】禁止定义在 components 中的组件未在 template 使用
13) 【强制】模版中已定义的变量必须使用
14) 【强制】<component> 必须有绑定的组件
15) 【强制】render 函数必须有返回值
16) 【强制】禁止在模版中用 this
17) 【强制】在 v-on 指令中，禁止调用没有参数的函数
18) 【强制】当一个节点上出现两个 v-on:click 时，其中一个必须为 exact
19) 【强制】不要把 v-if 和 v-for 同时用在同一个元素上（大部分时候你可以使用计算属性实现）

```
// not good
<ul>
  <li
    v-for="user in users"
    v-if="user.isActive"
```

```
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>

<ul>
  <li
    v-for="user in users"
    v-if="shouldShowUsers"
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>

// good
<ul>
  <li
    v-for="user in activeUsers"
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>

<ul v-if="shouldShowUsers">
  <li
    v-for="user in users"
    :key="user.id"
  >
    {{ user.name }}
  </li>
</ul>
```

20) 【强制】自闭合组件在单文件组件、字符串模板和 JSX 中没有内容的组件应该是自闭合的；但在 DOM 模板里不要这样做

```
// not good
<!-- 在单文件组件、字符串模板和 JSX 中 -->
<MyComponent></MyComponent>
<!-- 在 DOM 模板中 -->
<my-component/>

// good
<!-- 在单文件组件、字符串模板和 JSX 中 -->
```

```
<MyComponent/>
<!-- 在 DOM 模板中 -->
<my-component></my-component>
```

21) 【强制】模版中的组件名大小写在单文件组件和字符串模板中组件名应该总是 PascalCase 的；但是在 DOM 模板中总是 kebab-case 的

```
// not good
<!-- 在单文件组件和字符串模板中 -->
<mycomponent></mycomponent>
<!-- 在单文件组件和字符串模板中 -->
<myComponent></myComponent>
<!-- 在 DOM 模板中 -->
<MyComponent></MyComponent>

// good
<!-- 所有地方 -->
<my-component></my-component>
```

22) 【强制】JS/JSX 中的组件名（name 属性）应该始终是 PascalCase 的

```
// not good
Vue.component('myComponent', {
  // ...
})
import myComponent from './MyComponent.vue'
export default {
  name: 'myComponent',
  // ...
}
export default {
  name: 'my-component',
  // ...
}

// good
Vue.component('MyComponent', {
  // ...
})
Vue.component('my-component', {
  // ...
})
import MyComponent from './MyComponent.vue'
export default {
  name: 'MyComponent',
  // ...
}
```

23) 【强制】非空 HTML 特性值应该始终带引号
24) 【强制】可简写指令需要缩写 (用 : 表示 v-bind: 和用 @ 表示 v-on:)
25) 【强制】不要把一个变量作为多个且同时存在的组件的 v-model 绑定值
26) 【强制】组件模板应该只包含简单的表达式，复杂的表达式则应该重构为计算属性或方法

```
// not good
{{
  fullName.split(' ').map(function (word) {
    return word[0].toUpperCase() + word.slice(1)
  }).join(' ')
}}

// good
<!-- 在模板中 -->
{{ normalizedFullName }}
// 复杂表达式已经移入一个计算属性
computed: {
  normalizedFullName: function () {
    return this.fullName.split(' ').map(function (word) {
      return word[0].toUpperCase() + word.slice(1)
    }).join(' ')
  }
}
```

27) 【推荐】prop 名大小写，在声明 prop 的时候，其命名应该始终使用 camelCase，而在模板和 JSX 中应该始终使用 kebab-case

```
// not good
props: {
  'greeting-text': String
}
<WelcomeMessage greetingText="hi"/>

// good
props: {
  greetingText: String
}
<WelcomeMessage greeting-text="hi"/>
```

28) 【推荐】多个特性的元素应该分多行撰写，每个特性一行（此项 Vetur 插件会自动根据行宽阈值进行自动折行处理，一般无需考虑）

```
// not good
<img src="https://vuejs.org/images/logo.png" alt="Vue Logo">
<MyComponent foo="a" bar="b" baz="c"/>

// good
```

```
<img
  src="https://vuejs.org/images/logo.png"
  alt="Vue Logo"
>
<MyComponent
  foo="a"
  bar="b"
  baz="c"
/>
```

29) 【推荐】使用 import 或者 require 引入文件时，应该要带上文件扩展名

```
// not good
import User from "@/components/User"

// good
import User from  "@/components/User.vue"
```

30) 【推荐】应该把复杂计算属性分割为尽可能多的更简单的属性

```
// not good
computed: {
  price: function () {
    var basePrice = this.manufactureCost / (1 - this.profitMargin)
    return (
      basePrice -
      basePrice * (this.discountPercent || 0)
    )
  }
}}

// good
computed: {
  basePrice: function () {
    return this.manufactureCost / (1 - this.profitMargin)
  },
  discount: function () {
    return this.basePrice * (this.discountPercent || 0)
  },
  finalPrice: function () {
    return this.basePrice - this.discount
  }
}
```

4.4.4 组件/实例的选项的顺序

1) 【推荐】组件/实例的选项应该有统一的顺序，这是我们推荐的组件选项默认顺序：

a. 副作用 (触发组件外的影响)

- el

b. 全局感知 (要求组件以外的知识)

- name
- parent

c. 组件类型 (更改组件的类型)

- functional

d. 模板修改器 (改变模板的编译方式)

- delimiters
- comments

e. 模板依赖 (模板内使用的资源)

- components
- directives
- filters

f. 组合 (向选项里合并属性)

- extends
- mixins

g. 接口 (组件的接口)

- inheritAttrs
- model
- props/propsData

h. 本地状态 (本地的响应式属性)

- data
- computed

i. 事件 (通过响应式事件触发的回调)

- watch

j. 生命周期钩子 (按照它们被调用的顺序)

- beforeCreate
- created
- beforeMount
- mounted
- beforeUpdate
- updated
- activated
- deactivated
- beforeDestroy
- destroyed

k. 非响应式的属性 (不依赖响应系统的实例属性)

- methods

l. 渲染 (组件输出的声明式描述)

- template/render

- renderError

4.4.5 元素特性的顺序

1) 【推荐】元素（包括组件）的特性应该有统一的顺序，推荐的默认顺序：
   a. 定义 (提供组件的选项)
      - is
   b. 列表渲染 (创建多个变化的相同元素)
      - v-for
   c. 条件渲染 (元素是否渲染/显示)
      - v-if
      - v-else-if
      - v-else
      - v-show
      - v-cloak
   d. 渲染方式 (改变元素的渲染方式)
      - v-pre
      - v-once
   e. 全局感知 (需要超越组件的知识)
      - id
   f. 唯一的特性 (需要唯一值的特性)
      - ref
      - key
      - slot
   g. 双向绑定 (把绑定和事件结合起来)
      - v-model
   h. 其它特性 (所有普通的绑定或未绑定的特性)
   i. 事件 (组件事件监听器)
      - v-on
   j. 内容 (覆写元素的内容)
      - v-html
      - v-text

4.4.6 单文件组件的顶级元素的顺序

1) 【强制】单文件组件按照 <template>、<script> 和 <style> 的标签顺序

```
<!-- good -->
<!-- ComponentA.vue -->
<template>...</template>
<style>/* ... */</style>

<!-- ComponentB.vue -->
<template>...</template>
```

```
<script>/*
 * vue 结构
 * export default {
 * name: 当前文件名 --页面组件 name
 * data: 页面(组件)局部属性，必须 return 一个对象
 * props: 页面(组件)外部属性，定义尽可能详细，尽量定义类型、默认值等
 * components: 页面(组件)依赖非公共组件
 * mixins: 页面(组件)混入
 * computed: 页面(组件)计算属性
 * watch: 页面(组件)监听变量变化
 * created: 页面(组件)实例初始化完成
 * mounted: 页面(组件)渲染完成
 * destroyed: 页面(组件)销毁
 * methods: 页面(组件)方法
 */</script>
<style>/* ... */</style>
```

4.5 前端请求调用规范

请求调用是系统前端开发过程中较为常见的场景， 一般情况下，开发者会倾向基于现有的三方库来实现请求调用，比如 Vue.js 官方就推荐基于 Axios 作为统一的请求调用库。但是，三方库的引入会给系统开发带来一定的风险和不确定性，并且自身存在漏洞的三方库也无法通过公司内部的依赖扫描。

此外，三方库提供的接口可能过于宽泛，同样的功能基于不同的接口实现时会存在一定的代码差异，在这种情况下，当我们需要对请求调用的过程进行统一的干预和处理时，比如在全链路需求当中需要在请求调用时传入统一的 trace_id 字段，不同的代码实现就需要分别进行处理，不仅增加了开发工作量，也不便于研发过程的管理。

本章节对基于 HUI 框架进行前端开发的业务系统，约定了对应规范，用于规避上述提到的各种问题。

1) 【强制】业务系统开发过程当中应该优先基于 hCore 提供 fetch 接口完成或封装业务请求的调用方法。

说明：应用框架 hCore 基于 Axios 封装了统一的请求调用接口，一方面对三方依赖做了统一管理，便于进行维护和升级；另一方面，简化了请求配置，对请求的返回值和异常信息做了统一处理，支持 T2/T3 协议的应答适配，支持请求日志回溯；还基于中间件技术实现了对请求调用的扩展，并提供了一系列业务通用的中间件。开发者可以专注于业务的实现，在一定程度上提高了编码效率和代码质量，也便于后续的迭代升级。hCore 调用示例见 http://hui.hundsun.com/frame-api/。

2) 【参考】业务系统应该封装和维护独立的请求调用方法，合理定义出入参，避免与业务模块产生耦合。

3) 【参考】业务系统应该从权限校验的维度来构建 fetch 实例。一般地，当不同请求的权限校验逻辑一致时应该复用相同的 fetch 实例。

说明：请求调用的过程中往往要经过一定的权限校验，典型的比如会话权限的校验和用户权限的校验等等。权限校验的逻辑往往是业务无关的，在不同的业务请求过程当中可以复用。应用框架 hCore 提供的 fetch 接口支持多实例，同一个 fetch 实例应该对应一致的权限校验逻辑，便于统一维护。

4) 【推荐】业务系统应当基于请求中间件扩展 fetch 实例的功能范围，比如扩展默认配置，请求异常处理等。

5) 【推荐】业务系统请求中间件应当遵循职责单一原则，即一个中间件应当只做一件事情，避免中间件的功能耦合。

```
// 系统全局
import hCore from 'hui-core'

const app = hCore();

app.middleware('before-request-send', (config, next) => {
  // do something with config
  return next()
});

app.middleware('after-request-send', (err, res, next) => {
  if (err) {
    /* do something with err */
    return next()
  }

  /* do something with res */
  return next()
});

app.start();
```

5 文档修改记录

修订日期	修订说明	修订人	修订部门
2019/09/18	新增 HUI 前端开发规范	王亚男	EPG 代码组

2021/01/05	更新形成通用前端编码规范	李文博	EPG 研发组
2022/05/30	删除 4.2.2 下的第 23，4.4.3 下的第 9 条 修改 4.3.2 下的第 37 条的描述，4.3.2 下的第 58 条（补充例子说明），4.3.3 下第 3 条的描述，4.4.3 下第 5、6 条的规范等级（由强制改成推荐），4.2.2 下的第 51 条（示例代码调整） 4.1.9 下增加规范 【推荐】中英文之间、中文和数字之间用空格隔开。 4.1.1 下增加规范【推荐】文件或类中的私有属性，变量和方法名应该以下划线 "_" 开头；保护属性，变量和方法名不需要下划线开头，和公共变量名一样。建议结合 ES6 的私有属性规范，用 # 作为类的私有属性修饰符。	李文博	EPG 研发组
2022/10/20	增加 4.5 前端请求调用规范	顾将来	EPG 研发组